

Laborator 3

STRUCTURI ARITMETICE DE ÎMPĂRȚIRE

În acest laborator sunt studiate circuitele aritmetice de împărțire utilizate în unitățile de calcul. Pe parcursul acestuia se vor prezenta structurile componente ale circuitelor aritmetice de împărțire. O parte din acestea sunt similare ca la circuitul de înmulțire prin urmare au fost prezentate acolo. Circuitele de memorare de tip registru paralel – serie cu deplasare stânga dreapta au fost tratate amănunțit în cadrul laboratorului de Calculatoare numerice, din acest motiv aici se va prezenta sumar arhitectura și codul în limbajul VHDL. La fel ca și la algoritmul de înmulțire, se va prezenta generic algoritmul de împărțire secvențială, implementarea acestuia pe un circuit hardware (împărțire a două numere pe 4 biți fiecare) și în final secvența de comenzi pentru împărțire.

1. Registrul serie – paralel cu deplasare stânga dreapta

Alături de registrul cu acces paralel, în care este încărcat împărțitorul, de circuitul diferență (compus din sumator și rețea de inversoare) un circuit necesar care intră în componența sistemului de împărțire este registrul cu încărcare paralelă și deplasare serială în ambele sensuri (stânga dreapta) în care se va memora în final restul împărțirii.

Pentru deplasare în ambele sensuri sunt folosite multiplexoare 4:1 (sunt necesare trei intrări selectate) conform figurii de mai jos (figura 1).

Selecția direcției de deplasare se face folosind intrarea „s40_not04”. Când aceasta este 0 deplasarea se face de la O0 la O4, O0 fiind înlocuit cu valoarea care se află pe intrarea „si_bcps”. Când „s40_not04” este 1 deplasarea se face de la Q4 la Q0, Q4 fiind înlocuit cu valoarea din intrarea „si_bcms”. Aceste deplasări au loc numai dacă intrarea „s_notp” este 1. Dacă această intrare este 0 circuitul va acționa ca un registru paralel iar intrarea „s40_not04” este ignorată.

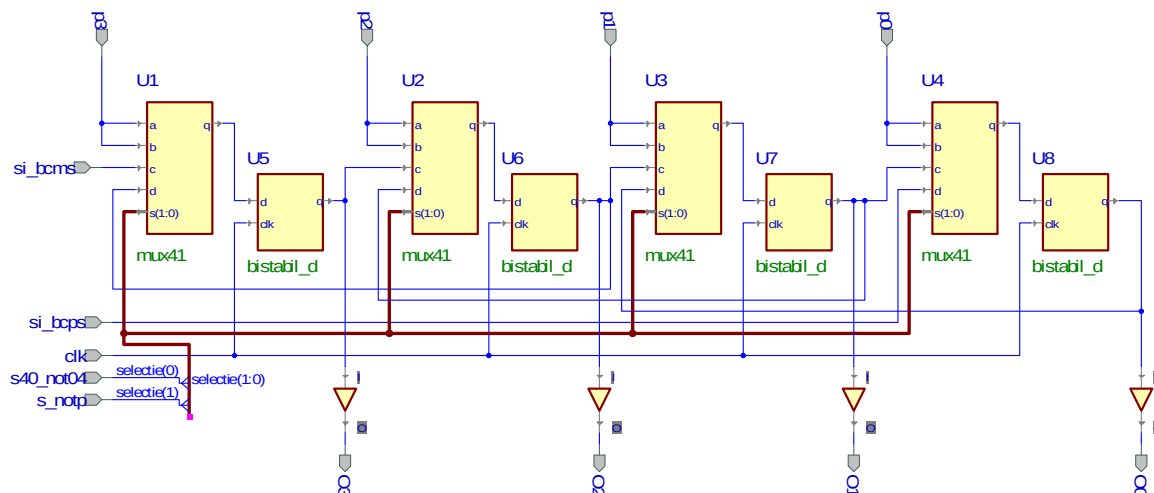


Fig. 1. Registrul serie paralel cu deplasare stânga dreapta

Descrierea comportamentală a registrului cu acces paralel și deplasare serie la stânga și la dreapta este prezentată mai jos:

```

library IEEE;
use IEEE.STD_LOGIC_1164.all;

entity rpsb4 is
  port(
    si_bcps : in STD_LOGIC;
    si_bcms : in STD_LOGIC;
    s03_not30 : in STD_LOGIC;
    s_notp : in STD_LOGIC;
    clk : in STD_LOGIC;
    p : in STD_LOGIC_VECTOR(3 downto 0);
    q : out STD_LOGIC_VECTOR(3 downto 0)
  );
end rpsb4;

architecture rpsb4_a of rpsb4 is
begin

  process(clk,si_bcps,si_bcms,s03_not30,s_notp,p)
    variable q_int : std_logic_vector(3 downto 0);
  begin
    if clk'event and clk='1' then
      if s_notp = '0' then
        q_int := p;
      else
        if s03_not30 = '0' then
          q_int(2 downto 0) := q_int(3 downto 1);
          q_int(3) := si_bcms;
        else
          q_int(3 downto 1) := q_int(2 downto 0);
          q_int(0) := si_bcps;
        end if;
      end if;
    end if;
    q <= q_int;
  end process;
end rpsb4_a;

```

2. Circuitul de împărțire pe 4 biți folosind al treilea algoritm de împărțire

a. Al treilea algoritm de împărțire

Algoritmul utilizează trei regiștri pentru stocarea operanzilor, a rezultatelor intermediare și a rezultatului final (rest și cât). Aceștia sunt: un registru cu încărcare paralelă pentru înmulțitor, un registru cu încărcare paralelă și deplasare la stânga pentru deînmulțit, rezultate parțiale și cât, un registru cu încărcare paralelă, deplasare la stânga și la dreapta pentru rezultate parțiale și rest. Inițial, registrul cu încărcare paralelă va conține împărțitorul (care va rămâne neschimbat pe tot parcursul algoritmului), registrul care va conține în final restul trebuie inițializat cu valoarea 0 în timp ce registrul care va conține în final câtul va fi inițializat cu deîmpărțitul.

Deoarece operația de deplasare la stânga se face la registrul rest și la registrul cât aceștia vor fi tratați pe parcursul algoritmului ca fiind concatenați într-un registru denumit registru rest – cât.

În prima etapă este deplasat la stânga registrul rest-cât (registru conține inițial deîmpărțitul). Este scăzut restul din împărțitor, rezultatul se depune în rest. Dacă rezultatul este negativ, restul este restaurat efectuând o adunare între rest și împărțitor. În acest caz restul și câtul sunt deplasate la stânga punând 0 pe ultima poziție. Dacă rezultatul scăderii este pozitiv, restul și câtul sunt deplasate la stânga punând 1 pe ultima poziție. Numărul de iterații la acest algoritm este egal cu numărul de biți care ai are deîmpărțitul. Algoritmul se încheie cu deplasarea la dreapta a registrului rest. În figura 2 este prezentată organigrama algoritmului 3 de împărțire. Avantajele folosirii acestui algoritm sunt, ca și la înmulțire, numărul mic de iterații și numărul mic de resurse hardware folosite. În acest caz, registrul „cât” joacă rol și pe post de menținere temporară a deîmpărțitului.

b. Circuitul de împărțire

Componentele folosite de circuitul de împărțire sunt următoarele:

- circuit diferență pe patru biți;
- registru paralel – paralel pe 4 biți;
- doi regiștrii serie – paralel cu deplasare stânga dreapta

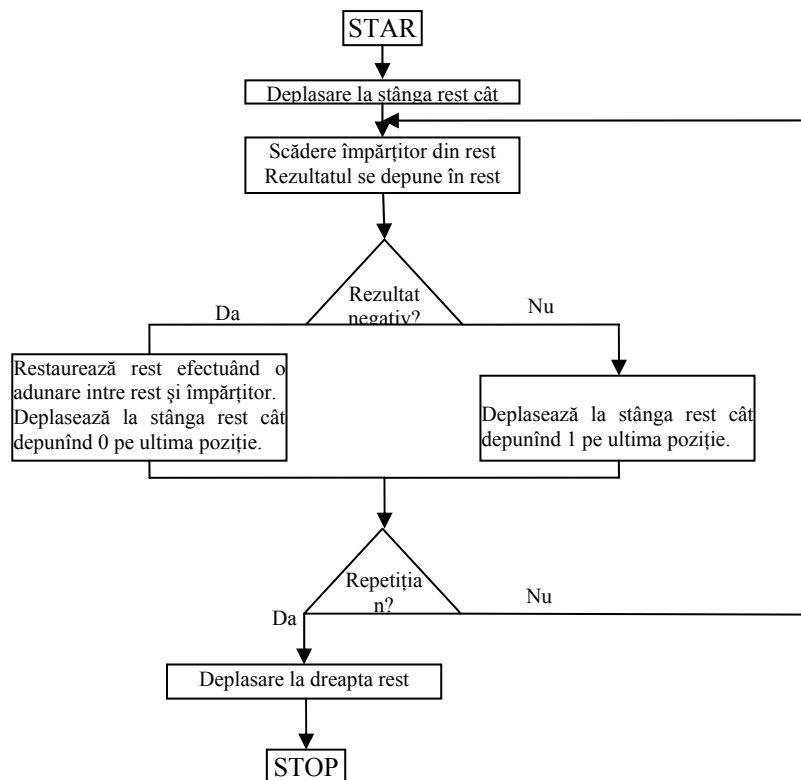


Fig. 2. Organigrama algoritmului 3 de împărțire

În figura de mai jos este prezentată schema circuitului de împărțire. Față de algoritmul de împărțire folosit mai sus au fost aduse niște modificări, pentru simplificarea schemei. Astfel:

- scăderea se face la fiecare iterație. Dacă semnul scăderii este pozitiv rezultatul este depus în registrul rest. Astfel se elimină un pas care necesita restaurarea restului. În aceste condiții, circuitul de adunare-scădere va fi permanent în modul scădere;
- registrul în care se depune deîmpărțitul și în care se va găsi în final câtul trebuie să efectueze o operație de încărcare paralelă și mai multe operații de deplasare la stânga. Se folosește un registru serie paralel cu deplasare stânga dreapta dar intrarea care specifică sensul de deplasare se va ține permanent în 1 (VCC) deci sensul de deplasare va fi permanent la stânga;
- circuitul de scădere este un circuit care generează numere cu semn. Semnul este dat de bitul 5.

Pentru circuitul hardware reprezentat în figura 3 pinii de intrare și ieșire utilizați sunt următorii:

Denumire pini	Direcționalitate	Explicație
I3..I0	Intrări	Intrări împărțitor
D3..D0	Intrări	Intrări deîmpărțit
Clk_cat	Intrare	Intrare ceas registru deîmpărțit – cât
Ser_notld	Intrare	Intrare configurare registru cât: deplasare cât sau încărcare deîmpărțit
Bi	Intrare	Bit cel mai puțin semnificativ (0 sau 1)
Clk_rest	Intrare	Ceas registrul rest
Clk_imp	Intrare	Ceas registru împărțitor
Ser_notdif	Intrare	Configurare registru rest: deplasare sau încărcare diferență
St_notdr	Intrare	Configurare sens deplasare registru rest
Cât3..cat0	Ieșiri	Ieșiri cât
Rest3..rest0	Ieșiri	Ieșiri rest
Semn	Ieșire	Ieșire semn pentru test

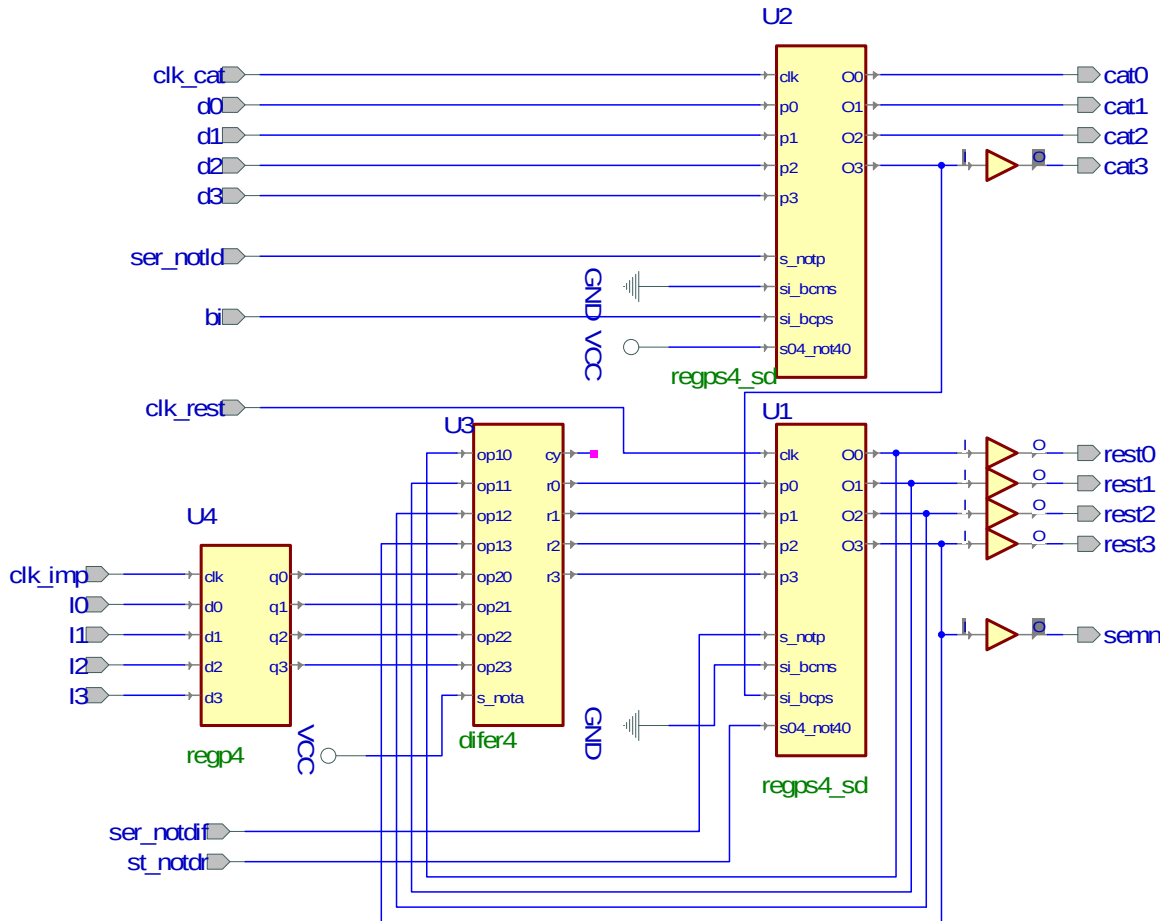


Fig. 3. Schema circuitului de împărțire

În concluzie, algoritmul reprezentat în figura 2 se va scrie astfel, utilizând comenzi la circuitul din figura 3:

- se vor inițializa circuitele următoare: U4 (registru cu acces paralel) cu împărțitorul (front pozitiv pe intrarea clk_imp), U2 (registru cu acces paralel și deplasare serie în ambele sensuri) cu deîmpărțitul (front pozitiv pe intrarea clk_cat având grijă ca intrarea ser_notld să fie 0 logic), U1 (registru cu acces paralel și deplasare serie în ambele sensuri) cu 0 (se forțează ieșirea circuitului diferență în 0 iar apoi se încarcă paralel registrul prin generarea unui front crescător pe intrarea clk_rest cu intrarea ser_notdif în 0 logic);
- se deplasează registrul rest cât la stânga (intrările ser_notdif și ser_notld în 1 logic, intrarea st_notdr în 1 logic și fronturi pozitive pe clk_rest și clk_cat, în această ordine!);
- se testează dacă semnul scăderii este pozitiv, adică, bitul 3 din rezultatul de la scădere este 0 (numerele sunt pe 4 biți cu semn!);
- în caz afirmativ, se încarcă diferența în registrul rest (ser_notdif în 0 pentru modul paralel și clk_rest) după care se deplasează la stânga registrul rest cât punând pe ultima poziție - în registrul cât ultimul bin cel mai puțin semnificativ - valoarea 1 (ser_notdif în 1 pentru modul serie, intrarea bi în 1 logic – intrarea serială pentru

- registrul cât atunci când acesta este deplasat la stânga – și apoi fronturi pozitive pe `clk_rest` și `clk_cat`);
- dacă semnul scăderii nu este pozitiv (adică bitul 3 rezultat scădere este 1) atunci se va face doar o deplasare la stânga în registrul `rest` – cât punând ultimul bit c.m.p.s în 0 (deci cu `ser_notdif` în 1 pentru modul serie și intrarea bi în 0 sunt generate fronturi pozitive pe `clk_rest` și `clk_cat`);
 - testarea, depunere rezultat diferență – dacă este cazul și deplasarea se repetă de 4 ori;
 - în final se deplasează registrul `rest` la dreapta cu o poziție (`ser_notdif` în 1, `st_notdr` în 0 – pentru setarea sensului de deplasare la dreapta și front crescător pe `clk_rest`).

Desfășurarea lucrării:

1. Se va proiecta un circuit de împărțire a două numere pe patru octeți. Pentru aceasta se vor urmări pașii următori:
 - a. Se va construi un proiect nou în ACTIVE HDL, și se va configura astfel încât să permită implementarea pe machetele de laborator cu Xilinx Spartan 3 (vezi anexa de laborator);
 - b. Se va proiecta un circuit diferență pe patru biți (cu transport anticipat sau transport parțial) folosind cunoștințele cumulate din laboratorul 1 (subiect evaluare laborator anterior);
 - c. Se vor construi regiștrii paralel – paralel și paralel – serie cu deplasare stânga dreapta.
 - d. Folosind circuitul diferență proiectat la punctul a., registrul paralel – paralel și registrul serie paralel se va realiza schema circuitului de împărțire a două numere pe patru biți (figura 3). Circuitele buffer necesare pentru separarea ieșirilor se vor adăuga din clasa Built In Symbols .
 - e. Pentru a verifica funcționalitatea circuitului, ieșirile acestuia se vor conecta la un decodor 7 segmente, cum se vede în figura de mai jos. Intrările cu împărțitorul și deîmpărțitul vor fi stabilite prin plasarea la valorile corespunzătoare în timp ce intrările de comenzi de tip `clk` se conectează la butoane prin interfețe (care asigură eliminarea oscilațiilor la comutarea butonului). Intrările care stabilesc direcționalitatea regiștrilor serie paralel vor fi conectate la comutatori (`sw_dip`).
Decodorul împreună cu circuitele de interfațare a butoanelor sunt descrise mai jos în limbajul VHDL.

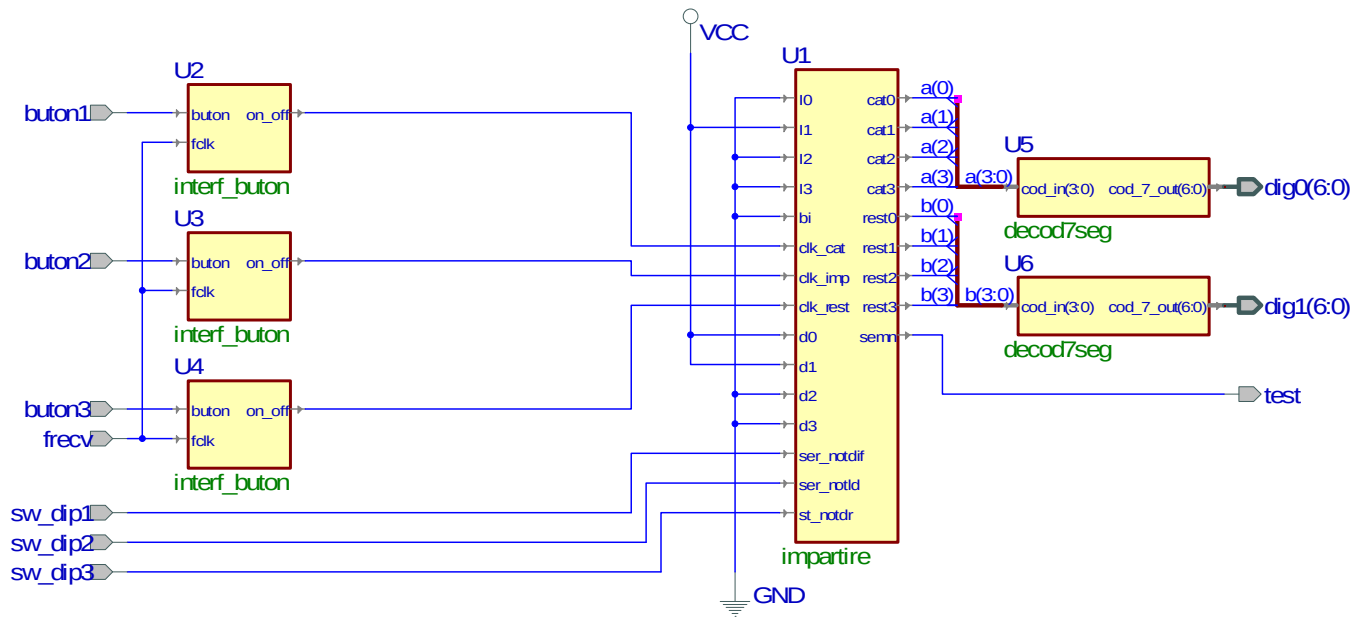


Fig. 11. Schema de testare a circuitului de împărțire

Se va construi, sintetiza și implementa structura din figura 10 (se urmăresc pașii specificați în anexa). Pentru conectarea intrărilor și ieșirilor la butoanele respectiv afișoarele de pe machetă se vor realiza următoarele alocări de pini:

Pin alocat	Pin fizic
Buton1	D1
Buton2	C1
Buton3	B6
Sw_dip1	Y6
Sw_dip2	V6
Sw_dip3	U7
Test	W2
Dig1(0)	C6
Dig1(1)	B8
Dig1(2)	E7
Dig1(3)	C5
Dig1(4)	E6
Dig1(5)	B5
Dig1(6)	A4
Dig0(0)	B10
Dig0(1)	A12
Dig0(2)	C10
Dig0(3)	A9
Dig0(4)	B9
Dig0(5)	A10
Dig0(6)	E9

- f. Pentru operanzii introduși se vor parcurge pașii care realizează împărțirea celor doi operanzi. Bitul de test (bitul de semn) este conectat separat ca ieșire la LED-ul 0 de pe machetă, acesta prezentând starea în funcție de care se va lua decizia în algoritm pentru încărcarea diferenței.

Decodorul binar – 7 segmente:

```

---decodorul binar - 7 segmente
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity decod7seg is
    Port (    cod_in : in std_logic_vector(3 downto 0);
            cod_7_out: out std_logic_vector(6 downto 0)
          );
end decod7seg;

architecture decod7seg_a of decod7seg is
begin
    process(cod_in)
    begin
        case cod_in is
            when "0000" => cod_7_out<= "0111111";
            when "0001" => cod_7_out<= "0000110";
            when "0010" => cod_7_out<= "1011011";
            when "0011" => cod_7_out<= "1001111";
            when "0100" => cod_7_out<= "1100110";
            when "0101" => cod_7_out<= "1101101";
            when "0110" => cod_7_out<= "1111101";
            when "0111" => cod_7_out<= "0000111";
            when "1000" => cod_7_out<= "1111111";
            when "1001" => cod_7_out<= "1101111";
            when "1010" => cod_7_out<= "1110111";
            when "1011" => cod_7_out<= "1111100";
            when "1100" => cod_7_out<= "0111001";
            when "1101" => cod_7_out<= "1011110";
            when "1110" => cod_7_out<= "1111001";
            when "1111" => cod_7_out<= "1110001";
            when others => cod_7_out<= "0000000";
        end case;
    end process;
end decod7seg_a;

```

Circuit interfață butoane (evitare debouncing):

```

----circuit interfata butoane
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

```



```

entity interf_buton is
  Port (      fclk : in std_logic;
          buton : in std_logic;
          on_off: out std_logic
        );
end interf_buton;

architecture interf_buton_a of interf_buton is
  signal      decnt : std_logic_vector(17 downto 0);
  signal onoff : std_logic;
begin
  --proces folosit pentru debouncing- interval de 5ms de temporizare
  deb : process (fclk,buton)
  begin
    if buton = '1' then
      decnt <= "000000000000000000";
      onoff <= '1';
    elsif fclk'event and fclk = '1' then
      if decnt < 250000 then
        decnt <= decnt + 1;
        onoff <= '1';
      else
        onoff <= '0';
      end if;
    end if;
  end process;
  on_off <= onoff;
end interf_buton_a;

```

Probleme. Întrebări:

1. Care este rolul circuitelor multiplexor Mux41 din registrul serie paralel din figura 1?
2. Care este numărul de pași executați la împărțirea a două numere pe 2 octeți folosind al treilea algoritm de împărțire?
3. Să se proiecteze un registru serie-paralel cu deplasare stânga-dreapta pe 8 biți.
4. Să se proiecteze un circuit de înmulțire pe 8 biți.
5. Să se proiecteze un circuit de împărțire pe 8 biți.

Grilă de evaluare (proiectare):

Activitate	Punctaj
Proiectare sumator (diferență) 4 biți	2
Proiectare registru paralel-paralel	1
Proiectare registru serie-paralel	1
Proiectare circuit înmulțire (împărțire) + schemă test	3
Implementare circuit test	3

